Deterministic Audit of LLM-Generated Research Claims

Vladimir V. Borovikov
Independent researcher
https://orcid.org/0009-0005-7773-3581

October 2026

Abstract

Large language models are increasingly used to draft the narrative parts of research: reports, verdicts, discussion sections. The failure mode that matters is not obvious fabrication but *plausible overstatement*: the number is right, the interpretation is not, and the prose is confident. We describe a protocol that makes such overstatement mechanically detectable, and report what happened when we applied it to our own corpus of 1,416 probes across ten experimental series. The audit covers ten p-values and twenty-one fractions, and Section 4 states its scope.

The protocol rests on six requirements: pre-registration freeze, external memory instead of context recall, separation of generator and auditor, deterministic parity checking of every reported number, a three-tier taxonomy of claim errors, and multi-model adversarial review. We contribute an executable checklist of twelve items and report its honest status on our own work: six pass, one is amended, one fails, two are partial, two require human judgement. The protocol is not fully applied, and the checklist says so.

1 Introduction

When an LLM writes up an experiment, the output is fluent by construction. The fluency is not evidence of correctness, but it reads as such—to reviewers, and to the authors. In our own series we observed a repeated pattern: the first draft of a verdict claimed more than the archive supported, and the claim was withdrawn only after a separate recount.

Examples from our logs, all verifiable in the public repository:

- a null result ($p = 0.338$) described as a refutation "with 100% strictness";
- a comparison of 2/72 against 0/72 reported as a "law" (Fisher $p = 0.4965$ [2]);
- a p-value printed as 4.81×10^{-13} where the exact two-sided Fisher test gives 2.909×10^{-18}—a tool error, not a model error, that survived six weeks;
- a footnote pointing to a data file that contained none of the cited bytes.

The last two are instructive. The third was produced by our own checker; the fourth by a model writing a confident sentence about a file it had not opened. Neither was caught by re-reading.

2 A taxonomy of claim errors

We distinguish three tiers, because they are caught by different means.

Tier 0 — reference fabrication. The text cites an object that does not exist: a file, a line, a dataset, a value. Numbers may be arithmetically perfect. Neither recomputation nor proofreading detects it, because there is nothing to recompute. Detection requires checking existence.

Tier I — interpretive overstatement. The archive supports the number, not the sentence. "$p = 0.34$" becomes "refuted". Repaired by reformulation.

Tier II — numerical error. The archive holds a different number than the text. Repaired only by recounting from the log.

Tier 0 is the most dangerous and the least defended: tiers I and II have mechanical checks, whereas tier 0 has historically relied on attention alone; the protocol adds an existence check for it (P5).

3 The protocol

P1. Freeze. Code, prompts, hypotheses and endpoints are hashed before the first probe. A self-signed manifest is a declaration, not a freeze: without a trusted timestamp it proves only the current state of the files. We introduce the status FREEZE-AMENDED for series whose files changed after the freeze but where the change is documented, the results were recomputed and matched, and regression tests passed. It differs from "annulled" in that the numbers survive, and from "frozen" in that byte-level pipeline reproduction is lost.

P2. External memory. Numbers are read from files, never recalled from context. A model that must remember will eventually remember wrongly.

P3. Separation of contours. The generator writes, the auditor counts. The auditor is independent only if it is written from a separate specification; being "a different script" is not independence.

P4. Parity check. Every reported number—p-values, fractions, percentages, confidence bounds—is recomputed by a script, and the script's output is published alongside the paper. Two additions proved essential in practice. First, a *known-answer test*: a deliberately corrupted reference that the checker must reject, otherwise a blind checker passes everything. Second, statistics must not be hand-rolled without cross-validation; a parity check that compares one homemade implementation with another compares a bug with a bug. Our own Fisher implementation carried an absolute tolerance of 10^{-12}, which at $p \approx 10^{-18}$ is 3.4×10^5 times larger than the value itself and inflated the tail by five orders of magnitude. It was found by an independent implementation, not by the protocol—which is why cross-validation is now a required step rather than a lucky accident.

P5. Taxonomy. As in Section 2, with Tier 0 checked by a script that extracts file paths and line ranges from a document and verifies that the targets exist.

P6. Multi-model review. Several model families review each other. We report this as the weakest pillar: the number of passes is a working convention, not a demonstrated optimum, and models trained on overlapping corpora err correlatedly, so an "ensemble" does not guarantee independence of errors.

4 Application and honest status

The checklist has twelve items and is executable. On our corpus it reports:

Status	Count	Meaning
Pass	6	checked mechanically
Amended	1	post-freeze edit, documented and recomputed
Fail	1	verifier independence: same author wrote both
Partial	2	pre-registration, verdict versioning
Manual	2	trusted timestamp, archive immutability

The single failure is not repairable by code. It requires a second author writing the checker from a specification they did not write themselves. We state this rather than paper over it, because a checklist that can only be passed by misreporting is worse than no checklist.

The coverage of P4 is ten p-values—nine two-sample contrasts and the post-hoc five-bin test—twenty-one fractions, and all reported percentages including Clopper–Pearson bounds [1], recomputed by two independent scripts whose agreement is verified on every run.

5 Limitations

The audit sample is not random: we audited the series where an audit was performed. We make no claim about the series we did not audit.

The nine hypotheses were reconstructed from the manuscript, not frozen before the run. Multiple-comparison correction (Holm) [3] is discussed prospectively; it was not applied to the completed series.

Provenance is not provable: without a trusted timestamp we cannot show that the archive was in its present state *before* the run rather than adjusted after. What is demonstrated is narrower—the numbers reproduce from the archive as it exists in the repository today.

6 Availability

Code, probes and the executable checklist: https://github.com/skywhiteangels-cell/llm-token-architect. The accompanying study: https://doi.org/10.5281/zenodo.23679671.

Model identity. All probe series reported here ran on `flash_lite`, a publicly available model tier; the verdict texts were written with a different general-purpose model, noted in the project archive. Nothing is withheld behind a proprietary claim or a non-disclosure agreement: the archive and the verification scripts are released under the MIT licence, so every number can be recomputed without knowing which model produced it.

References

[1] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.

[2] R. A. Fisher. *The Design of Experiments*. Oliver & Boyd, Edinburgh, 1935.

[3] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.